

rppgExtractor.ts

frontend/signal-processing | TypeScript

```

/**
 * rppgExtractor.ts
 */
* Core rPPG signal pipeline – the lowest layer of the visual channel.
*
* This module is intentionally pure (no React, no DOM dependencies) so it
* can run inside a Web Worker for non-blocking signal processing.
*
* Input: Raw pixel data from the forehead ROI (ImageData.data array)
* Output: Smoothed green-channel value + derived stress component
*/
import { KalmanFilter, movingAverage, computeNormalizedVariance } from '../utils/kalmanFilter';
// ■■■ Types ■■■
export interface RPPGSample {
  rawGreen: number; // Average green pixel value (0-255)
  smoothed: number; // Kalman-filtered value
  timestamp: number; // Performance.now() at capture
}
export interface RPPGResult {
  sample: RPPGSample;
  varianceScore: number; // Normalized 0-100 (fed into fusion model)
}
// ■■■ Extractor class ■■■
export class RPPGExtractor {
  private kalman = new KalmanFilter(0.1, 1.0);
  private buffer: number[] = [];
  private baseline: number = 0;
  private calibrated = false;
  private readonly BUFFER_SIZE = 150;
  private readonly CALIBRATION_FRAMES = 90;
  /**
   * Process one video frame's worth of ROI pixel data.
   *
   * @param pixelData - Uint8ClampedArray from ctx.getImageData() (RGBA layout)
   * @returns RPPGResult with smoothed sample and variance score
   */
  process(pixelData: Uint8ClampedArray): RPPGResult {
    const rawGreen = this.extractGreenMean(pixelData);
    const smoothed = this.kalman.update(rawGreen);
    this.buffer.push(smoothed);
    if (this.buffer.length > this.BUFFER_SIZE) this.buffer.shift();
    // Calibration: establish personal baseline from first N frames
    if (this.buffer.length === this.CALIBRATION_FRAMES && !this.calibrated) {
      this.baseline = this.buffer.reduce((a, b) => a + b, 0) / this.buffer.length;
      this.calibrated = true;
    }
    const varianceScore = this.calibrated
      ? computeNormalizedVariance(this.buffer, this.baseline, 30)
      : 0;
    return {
      sample: { rawGreen, smoothed, timestamp: performance.now() },
      varianceScore: Math.round(varianceScore),
    };
  }
  /**
   * Extract mean green channel value from RGBA pixel array.

```

```

*
* RGBA layout: [R0, G0, B0, A0, R1, G1, B1, A1, ...]
* Green channel is every 4th element starting at index 1.
*/
private extractGreenMean(data: Uint8ClampedArray): number {
  if (data.length === 0) return 0;
  let sum = 0;
  for (let i = 1; i < data.length; i += 4) sum += data[i];
  return sum / (data.length / 4);
}

reset(): void {
  this.kalman = new KalmanFilter(0.1, 1.0);
  this.buffer = [];
  this.baseline = 0;
  this.calibrated = false;
}

get isCalibrated(): boolean { return this.calibrated; }
get bufferLength(): number { return this.buffer.length; }
}

```